Recently, I came across an unusual AI model called Jev. Its defining feature is that it does not generate free-form text. Instead, it handles what seem like simple multiple-choice questions, scoring tasks, and true-or-false judgments. To me, this felt like a move in the opposite direction from what I had come to expect. I had been looking for increasingly rich, human-like responses from AI, but Jev asks its users to define the possible answers in advance.

Its speed surprised me even more. According to benchmarks published by TypeSafe, Jev can be tens or even hundreds of times faster than the language models included in the comparison on certain structured decision tasks. That does not mean it is faster than every Flash model in every situation, but the results still made a strong impression on me.

As I learned more, I discovered that TypeSafe, the company behind Jev, drew inspiration from an idea presented in *Thinking, Fast and Slow* by Nobel Prize-winning psychologist Daniel Kahneman. System One refers to fast, intuitive thinking, emphasizing quick and focused judgments. System Two refers to slower, more deliberate thinking, involving careful analysis and reasoning. Jev is designed to take on the first of these roles.

This is easy to relate to everyday life. When I see Starbucks, I immediately think of coffee rather than soda or juice. When I see someone wearing a red top with green trousers, I might instinctively feel that the combination does not work. When I see people fighting, I immediately sense that something is wrong and that I should stay alert. That repeated idea of acting “instinctively” comes close to how I understand Jev’s core philosophy: making a quick judgment without having to explain every step along the way. Of course, intuition is not always right, and a quick decision is not necessarily a correct one.

This changed how I evaluate AI. I used to be impressed by how complete a model’s answer sounded. Now, I find myself asking whether it really needs to say so much in the first place. Sometimes, all we care about is whether the work gets done correctly. We do not need an explanation or a detailed reasoning process attached to every result. In those situations, response time and decision accuracy may matter more than how “intelligent” the answer sounds.

I find it helpful to think of Jev as “a traditional classifier combined with semantic understanding.” It can understand natural-language input, but what it delivers is a decision within a predefined answer space. In that sense, it feels like a small renaissance for the classifier. Classification itself has not suddenly become new; rather, we are reconsidering the role it can play in an intelligent system. For example, Jev could first decide whether something should be done and what should be done, while an LLM works out how to do it, instead of leaving everything to the same model.

This also reminded me of decision trees and binary search. When facing a complex task, we do not necessarily have to begin by asking, “How should this entire problem be solved?” We could first ask, “Which category does it belong to?” and then, “Does it meet this condition?” A series of smaller questions can gradually narrow the possibilities. A complex task does not always require one equally complex answer.

It even reminded me of the lateral-thinking game known in Chinese as “Turtle Soup.” The host may respond only with “yes,” “no,” or “irrelevant,” yet the players can still uncover the full story through a sequence of questions. The length of an answer does not determine its value. What matters more is whether the question and its answer help us move forward.

Another aspect that interested me was probability. Ordinary generative language models already rely on probabilities when producing tokens. Jev’s distinction is not that it uses probabilities while other models do not. Rather, it returns probabilities for the candidate answers directly to software, instead of constructing a response one token at a time. To me, this shifts the focus from “What did the model ultimately say?” to “How did it weigh the possible answers?”

Once I understood these ideas, Jev’s applications became much less abstract. One of the most interesting uses, in my view, is evaluation and filtering. Before asking a model to answer a question, we could use Jev to assess which materials are relevant, which provide usable evidence, and which conflict with the assumptions in the question. The system can then decide how to pass those materials to the next model. Jev does not write the final answer, but it can help determine what that answer should be based on. This made me realize that a model’s value may lie not only in how much content it produces, but also in how much unsuitable content it helps a system filter out.

Ultimately, I became more convinced that learning from ourselves remains a worthwhile direction in the search for artificial intelligence. Neural networks, attention mechanisms, and System One naturally make me think of neural connections, the allocation of attention, and intuitive judgment. I do not see these connections as simple attempts to copy the human brain. Instead, I see them as sources of inspiration: understanding how we think can also help us think about how to design intelligent systems.

This also sparked my interest in the broader idea behind specialized LLMs. Strictly speaking, Jev is not a specialized LLM designed to generate language; it is more accurately described as an AI model built for decision-making. What it taught me is that a model does not have to be good at everything. It can be valuable simply because it has a clearly defined role within a larger system.

Following this line of thought, I also see more possibilities in domain-specific applications. Rather than starting with an agent that can “do everything,” it may be better to build one that clearly understands the specific job it needs to do well. Even when a system needs to generate content, it does not necessarily need complete freedom over its output. It could instead deliver only the scripts, action instructions, or fixed-format data required by the task.

For me, the next step is not just to learn how to make models better at answering questions. It is to learn how to give them the right responsibilities, so they can make useful decisions in real applications and deliver what is actually needed.